



## Results

0%

of text is likely AI ⓘ



AI-generated ⓘ 0%

Human-written &amp; AI-refined ⓘ 0%

Human-written ⓘ 100%



Caution: Our AI Detector is advanced, but no detectors are 100% reliable, no matter what their accuracy scores claim. Never use AI detection alone to make decisions that could impact a person's career or academic standing.

## 1. Introduction

This paper shows the deployment of Automated Security Testing with Static Application Security Testing (SAST) within a CI/CD pipeline. It aims to uncover the unsafe code used in a vulnerable web application based on Flask. This project aims to confirm whether tools such as Semgrep can detect any security vulnerabilities present in web applications. The application consists of components such as authentication of users, product management, and a review feature, which use an SQLite database. In this regard, the purpose here is to ascertain how effective the static code analysis tool is in detecting security vulnerabilities such as SQL injection, cross-site scripting (XSS) attacks, and authentication weaknesses.

## 2. Tool Selection and Rationale

Reasons that have motivated the choice of Semgrep are related to its simplicity, open-source status, and strong rules engine. This tool works very effectively with Python Flask applications and can help to identify any insecure coding techniques such as SQL injection, HTML rendering, and cookies. While black box approaches lack transparency in defining rules, Semgrep ensures that the developer understands why the flaw is being identified by providing transparency in defining its rules.

Furthermore, the smooth integration of Semgrep within CI/CD pipelines ensures that it works well in DevSecOps settings.

Two rule sets were used in this project:

- p/security-audit for high-confidence security issues

- p/python for broader vulnerability coverage

### 3. CI/CD Pipeline Implementation

The implementation of GitHub Actions for security scanning was done for every commit and pull request. Thus, any insecure code that may be committed will be identified immediately.

The pipeline performs the following steps:

- Checks out the repository
- Sets up a Python environment
- Installs Semgrep
- Executes a security scan using predefined rule sets

This is an application of the shift-left security approach in which vulnerabilities are detected during the development cycle rather than during production.

### 4. Static Analysis Results

Insecure cookie settings and debug mode of Flask application are two of the most reliable vulnerabilities discovered using p/security-audit. But p/python scanning found 46 more vulnerabilities.

#### Vulnerability Validation

The validity of SQL injection results was determined by looking at how user inputs were concatenated directly into the SQL queries using Python f-strings. For example, login credentials and cookie values are embedded directly into database queries without parameterisation. XSS vulnerabilities have also been verified in cases where user inputs are directly inserted into HTML code as part of user inputs such as reviews of products. This confirms that the identified flaws can be exploited in real-world environments.

Figure 4: SQL Injection, Authentication Query Vulnerability

Figure 5: Cross-Site Scripting (XSS) / HTML Injection

Figure 6: SQL Injection via Cookie-Based Authentication Bypass (Session Hijack Risk)

Figure 7: SQL Injection in UPDATE Query (Privilege Abuse / Account Escalation)

### 6. False Positives and Triage

Although most Semgrep results were accurate, static code analysis can potentially result in false positives because it lacks knowledge about runtime behavior. For example, even if there is a warning regarding the HTML format strings, they will not necessarily lead to an XSS attack if the user's input is properly sanitized elsewhere. In any case, the triage process was carried out manually by following the

path of data flow from its origin (form/cookies) up until it got to its destination (HTML). This highlights the importance of human validation alongside automated scanning.

#### 7. CI/CD Security Execution Evidence

The pipeline for GitHub Actions was run effectively, such that Semgrep was automated in each push action. The pipeline managed to install dependencies, perform static analysis, and report on vulnerabilities without any manual effort. This shows that continuous enforcement is in place, satisfying the CI/CD requirement of the assignment.

#### 8. Limitations of Static Analysis

Despite all this, there are some drawbacks that come along with Semgrep. One of these includes the fact that Semgrep only identifies vulnerabilities by analyzing predefined rules. Semgrep fails to recognize any vulnerability pattern that may not be listed in its pre-existing rules. Moreover, Semgrep is not context-aware; hence, it might fail to detect certain vulnerabilities while generating false positives.

#### 9. Security Engineering Insight

In conclusion, this activity shows that building secure applications involves the use of layered security measures. Static analysis methods like Semgrep prove to be extremely efficient when applied for identifying vulnerabilities at their early stages. However, they have to be used together with secure coding practices and code reviews, along with dynamic testing. The use of Semgrep in the CI/CD pipeline ensures that high levels of security are sustained.

#### 10. Conclusion

It is important to mention that the integration of Semgrep into the CI/CD pipeline using GitHub Actions was highly effective in conducting automated security testing for the application containing vulnerabilities. Nevertheless, there are numerous security flaws found by the mentioned software such as SQL injection, XSS, and weak authentication among others. Nonetheless, it should be noted that such analysis is subject to certain limitations like dependency on the rule set and absence of runtime information. Overall, this experience can act as proof that static code analysis is an integral part of DevSecOps operations that need to be complemented with other security techniques.